DATA STRUCTURES

CO2 Notes: Stack, Recursion and Queue (Unit II)

Course Code: PBC2TE302 | PBCA | Sem III

Course Outcome (CO2)

Master stack operations and their applications, including expression evaluation and problem-solving using recursion. Develop expertise in implementing different queue structures.

Topics covered in these notes

- Stack: terminology, representation in memory, PUSH / POP / PEEK operations
- Polish notations: infix, prefix, postfix and conversions
- Evaluation of postfix expressions
- Recursion and problems on recursion
- Applications of stack: Tower of Hanoi and Quick Sort
- Queue: memory representation, operations using array and linked list
- Circular queue, dequeue (double-ended queue), priority queue and applications
- Quick revision and practice questions

1. Stack

1.1 Definition and terminology

A **stack** is a **linear data structure** in which insertion and deletion are done **only at one end**, called the **TOP**. It follows the **LIFO (Last In First Out)** rule: the element inserted last is removed first. Real-life examples: a pile of plates, a stack of books, the Undo button in an editor, the Back button of a browser.

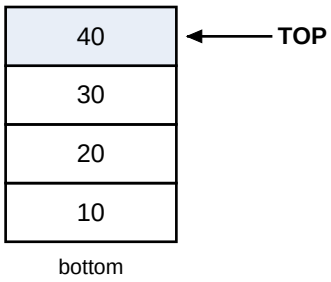


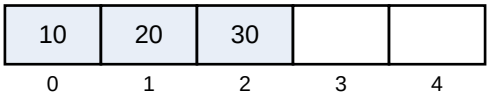
Fig 2.1: A stack holding 10, 20, 30, 40. 40 was pushed last so it will be popped first

Term	Meaning
TOP	Variable/pointer that holds the position of the last inserted element. Empty stack: top = -1.
PUSH	Operation to insert an element on the top of the stack.
POP	Operation to remove (and return) the top element.
PEEK / TOP element	Read the top element without removing it.
Stack Overflow	Trying to PUSH when the stack is already full.
Stack Underflow	Trying to POP when the stack is empty.
isEmpty / isFull	Functions to check whether the stack is empty (top = -1) or full (top = MAX-1).

1.2 Representation of a stack in memory

(a) Using an array (static)

A one-dimensional array **stack[MAX]** and an integer variable **top** are used. Elements are stored from index 0 and top is the index of the last element.



Array of size 5 with 3 elements: top = 2 (index of 30)

```
#define MAX 5
int stack[MAX];
int top = -1;      // stack is empty
```

(b) Using a linked list (dynamic)

Each element is a node. **Insertion and deletion are done at the beginning of the list**, so the head pointer works as TOP. No overflow problem (until the memory is full) and no memory wastage.

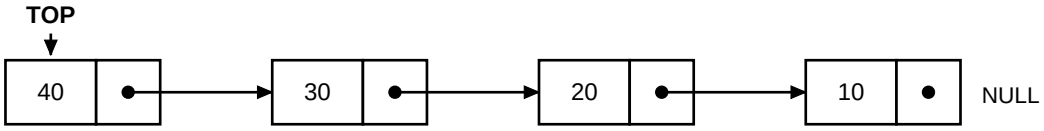


Fig 2.2: Stack using linked list - the first node is the top

1.3 Operations on a stack

PUSH operation

Algorithm: PUSH(stack, item)
Step 1: If top == MAX - 1, print "Stack Overflow" and stop.
Step 2: top = top + 1.
Step 3: stack[top] = item.
Step 4: Stop.

POP operation

Algorithm: POP(stack)
Step 1: If top == -1, print "Stack Underflow" and stop.
Step 2: item = stack[top].
Step 3: top = top - 1.
Step 4: Return item. Stop.

PEEK operation

Algorithm: PEEK(stack)
Step 1: If top == -1, print "Stack is empty" and stop.
Step 2: Return stack[top].

```
#include <stdio.h>
#define MAX 5
```

```
int stack[MAX], top = -1;

void push(int item) {
    if (top == MAX - 1) printf("Stack Overflow\n");
    else stack[++top] = item;           // increase top, then store
}
int pop() {
    if (top == -1) { printf("Stack Underflow\n"); return -1; }
    return stack[top--];                // return value, then decrease top
}
int peek() { return (top == -1) ? -1 : stack[top]; }
void display() { int i; for (i = top; i >= 0; i--) printf("%d\n", stack[i]); }

int main() {
    push(10); push(20); push(30);
    printf("%d popped\n", pop());       // 30 popped
    printf("Top = %d\n", peek());       // Top = 20
    return 0;
}
```

Trace:

Operation	Stack (bottom -> top)	top
PUSH 10	10	0
PUSH 20	10, 20	1
PUSH 30	10, 20, 30	2
POP (returns 30)	10, 20	1
PEEK (returns 20)	10, 20	1

Stack using linked list (idea)

- **push:** newnode->next = top ; top = newnode ;
- **pop:** if top == NULL underflow; else temp = top ; top = top->next ; free(temp) ;

1.4 Applications of stack

- Conversion of expressions (infix to postfix / prefix) and evaluation of postfix expressions.
- Recursion - function calls are managed using the call stack.
- Undo / Redo in editors, browser back button.
- Checking balanced parentheses ( ) { } [ ].
- Tower of Hanoi, Quick Sort (non-recursive version), Depth First Search of a graph.

2. Polish Notations (Expression Notations)

An arithmetic expression can be written in three ways depending on the **position of the operator**:

Notation	Position of operator	Example for A + B	Also called
Infix	Between the operands	A + B	Normal human notation
Prefix	Before the operands	+ A B	Polish notation
Postfix	After the operands	A B +	Reverse Polish notation

Why postfix/prefix? They do not need brackets and no precedence rules are required while evaluating. A computer can evaluate them very easily using a stack. In infix we need brackets and precedence rules, which is harder for the machine.

Infix	Prefix	Postfix
A + B	+ A B	A B +
A + B * C	+ A * B C	A B C * +

Infix	Prefix	Postfix
(A + B) * C	* + A B C	A B + C *
A * B + C * D	+ * A B * C D	A B * C D * +

Operator precedence and associativity

Priority	Operator	Associativity
Highest (3)	^ (power)	Right to left
Medium (2)	* / %	Left to right
Lowest (1)	+ -	Left to right

2.1 Translation of infix to postfix

Algorithm: INFIX to POSTFIX (using a stack of operators)

Scan the infix expression from **left to right**, one symbol at a time.

If the symbol is an **operand** (A, B, 5...): write it directly to the output.

If the symbol is **(** : push it on the stack.

If the symbol is **)** : pop operators from the stack to the output until (is found; discard both brackets.

If the symbol is an **operator**: while the top of the stack is an operator with **higher or equal** priority, pop it to the output (for ^ pop only if higher). Then push the current operator.

After the last symbol, pop all remaining operators from the stack to the output.

Example 1: Convert (A + B) * C into postfix

Symbol	Stack	Output (postfix)	Action
(	(		push (
A	(	A	operand -> output
+	(+	A	push +
B	(+	A B	operand -> output
)		A B +	pop until (
*	*	A B +	push *
C	*	A B + C	operand -> output
end		A B + C *	pop all

Postfix = A B + C *

Example 2: Convert A + B * C - D / E into postfix

Symbol	Stack	Output	Action
A		A	output
+	+	A	push
B	+	A B	output
*	+ *	A B	* is higher than +, so push
C	+ *	A B C	output
-	-	A B C * +	pop * and + (higher/equal), push -
D	-	A B C * + D	output
/	- /	A B C * + D	/ higher than -, push
E	- /	A B C * + D E	output
end		A B C * + D E / -	pop all

Postfix = A B C * + D E / -

Example 3 (answer only): A + (B * C - (D / E ^ F) * G) * H -> **A B C * D E F ^ / G * - H * +**

2.2 Translation of infix to prefix

Method

Step 1: **Reverse** the infix expression and change every (into) and every) into (.

Step 2: Convert this new expression into **postfix** (note: for equal priority operators, pop only if the stack operator has strictly higher priority).

Step 3: **Reverse** the result. That is the prefix expression.

Example: Convert (A + B) * C into prefix.

1. Reverse and swap brackets: C * (B + A)
2. Convert to postfix: C B A + *
3. Reverse: *** + A B C**

Check with A + B * C: reverse = C * B + A -> postfix = C B * A + -> reverse = **+ A * B C**.

2.3 Evaluation of a postfix expression

Algorithm: EVALUATE POSTFIX

Scan the postfix expression from **left to right**.

If the symbol is an **operand**: push it on the stack.

If the symbol is an **operator**: pop two elements. The first popped is **operand2**, the second popped is **operand1**. Compute **operand1 operator operand2** and push the result.

At the end, the single value left on the stack is the answer.

Example 1: Evaluate 5 3 + 2 *

Symbol	Action	Stack (bottom -> top)
5	push 5	5
3	push 3	5, 3
+	pop 3 and 5; 5 + 3 = 8; push 8	8
2	push 2	8, 2
*	pop 2 and 8; 8 * 2 = 16; push 16	16

Answer = 16

Example 2: Evaluate 2 3 1 * + 9 -

Symbol	Action	Stack
2, 3, 1	push all three	2, 3, 1
*	3 * 1 = 3	2, 3
+	2 + 3 = 5	5
9	push 9	5, 9
-	5 - 9 = -4	-4

Answer = -4. Note: order matters for - and /, always operand1 (deeper) first, then operand2 (top).

```
int evaluate(char *p) {           // single-digit operands only
    int stack[50], top = -1, a, b;
    for (int i = 0; p[i] != '\0'; i++) {
        if (p[i] >= '0' && p[i] <= '9') stack[++top] = p[i] - '0';
        else {
            b = stack[top--]; a = stack[top--];
```

```
            switch (p[i]) {
                case '+': stack[++top] = a + b; break;
                case '-': stack[++top] = a - b; break;
                case '*': stack[++top] = a * b; break;
                case '/': stack[++top] = a / b; break;
            }
        }
    }
    return stack[top];
}
```

3. Recursion

Recursion is a technique in which a **function calls itself** to solve a smaller version of the same problem. Every recursive function must have:

- **Base case:** the condition where the function stops calling itself (otherwise infinite recursion and stack overflow).
- **Recursive case:** the part where the function calls itself with a smaller input.

Example 1: Factorial

n! = n x (n-1)! and 0! = 1! = 1

```
int fact(int n) {
    if (n == 0 || n == 1) return 1;     // base case
    return n * fact(n - 1);             // recursive case
}
```

How it works (call stack) for fact(4):

Going down (calls)	Coming back (returns)
fact(4) = 4 * fact(3)	fact(4) = 4 * 6 = **24**
fact(3) = 3 * fact(2)	fact(3) = 3 * 2 = 6
fact(2) = 2 * fact(1)	fact(2) = 2 * 1 = 2
fact(1) = 1 (base case)	fact(1) returns 1

Role of stack in recursion

- Each function call is pushed on the **system (call) stack** with its local variables and return address.
- When the base case is reached, calls are popped one by one (LIFO) and results are returned.
- That is why a recursion without a base case causes **stack overflow**.

Example 2: Fibonacci series

```
int fib(int n) {
    if (n == 0) return 0;               // base cases
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);     // 0 1 1 2 3 5 8 ...
}
```

Example 3: Sum of first n natural numbers, GCD and Power

```
int sum(int n)  { if (n == 0) return 0; return n + sum(n - 1); }          // sum(4)=10
int gcd(int a, int b) { if (b == 0) return a; return gcd(b, a % b); }     // gcd(12,8)=4
int power(int x, int n) { if (n == 0) return 1; return x * power(x, n-1); } // 2^3 = 8
```

Recursion	Iteration (loop)
Function calls itself	Uses for / while loop
Code is short and clear	Code is longer
Uses more memory (call stack)	Uses less memory
Slower due to function call overhead	Faster

Recursion	Iteration (loop)
Needs a base case	Needs a loop condition

3.1 Application of stack: Tower of Hanoi

There are **three pegs (A = source, B = auxiliary, C = destination)** and n disks of different sizes placed on A with the largest at the bottom. Move all disks to C with these rules:

- Move only **one disk at a time**.
- Only the **top disk** of a peg can be moved.
- A **larger disk can never be placed on a smaller disk**.

Recursive idea: (1) move n-1 disks from A to B using C, (2) move the biggest disk from A to C, (3) move the n-1 disks from B to C using A. Number of moves = $2^n - 1$.

```
void hanoi(int n, char A, char B, char C) {   // A=source, B=auxiliary, C=destination
    if (n == 1) { printf("Move disk 1 from %c to %c\n", A, C); return; }
    hanoi(n - 1, A, C, B);
    printf("Move disk %d from %c to %c\n", n, A, C);
    hanoi(n - 1, B, A, C);
}
```

Moves for 3 disks (7 moves):

Move	Disk	From	To
1	1 (smallest)	A	C
2	2	A	B
3	1	C	B
4	3 (largest)	A	C
5	1	B	A
6	2	B	C
7	1	A	C

3.2 Application of stack: Quick Sort

Quick sort is a divide-and-conquer sorting method. It uses recursion (the stack) to sort sub-arrays.

1. Choose a **pivot** element (say the first or last element).
2. **Partition:** rearrange so that all smaller elements are on the left of the pivot and all greater elements on the right. The pivot is now at its correct final position.
3. Recursively apply quick sort on the left part and on the right part.

Example (pivot = last element): [8, 3, 7, 1, 5]

Step	Array	Explanation
1	8 3 7 1 **5**	pivot = 5
2	3 1 **5** 7 8	after partition: left {3,1}, pivot 5 in place, right {7,8}
3	1 3 **5** 7 8	sort left part {3,1} -> 1 3
4	1 3 5 7 8	right part {7,8} is already sorted

```
void quick(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);   // pivot goes to index p
        quick(a, low, p - 1);
        quick(a, p + 1, high);
    }
}
int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1, j, t;
    for (j = low; j < high; j++)
```

```
        if (a[j] < pivot) { i++; t = a[i]; a[i] = a[j]; a[j] = t; }
    t = a[i+1]; a[i+1] = a[high]; a[high] = t;
    return i + 1;
}
```

Time complexity: best / average case O(n log n), worst case O(n^2) (when the array is already sorted and the first/last element is chosen as pivot).

4. Queue

4.1 Definition and terminology

A **queue** is a **linear data structure** in which insertion is done at one end called **REAR** and deletion is done at the other end called **FRONT**. It follows the **FIFO (First In First Out)** rule: the element inserted first is removed first. Real-life examples: a line at a ticket counter, a printer queue, customers waiting at a bank.

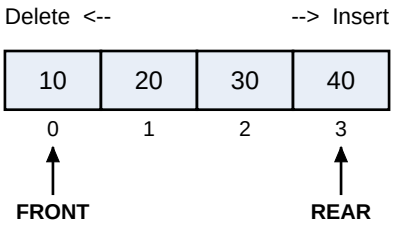


Fig 2.3: A queue with front at 10 and rear at 40

Term	Meaning
FRONT	Index/pointer of the first element (where deletion happens).
REAR	Index/pointer of the last element (where insertion happens).
ENQUEUE (Insert)	Add an element at the rear.
DEQUEUE (Delete)	Remove an element from the front.
Queue Overflow	Insert when the queue is full (rear = MAX - 1).
Queue Underflow	Delete when the queue is empty (front = -1 or front > rear).

Stack	Queue
LIFO	FIFO
Insertion and deletion at the same end (TOP)	Insertion at REAR and deletion at FRONT
Operations: push, pop	Operations: enqueue, dequeue
Used in recursion, expression evaluation	Used in scheduling, buffering, BFS

4.2 Memory representation of a queue

- **Using an array:** an array queue[MAX] with two variables front and rear (both initially -1).
- **Using a linked list:** each element is a node; **front** points to the first node and **rear** points to the last node. Insertion at the end, deletion from the beginning.

4.3 Queue operations using an array (Linear queue)

Insert (ENQUEUE)

Algorithm: INSERT(queue, item)

Step 1: If rear == MAX - 1, print "Queue Overflow" and stop.

Step 2: If front == -1, set front = 0 (first element being inserted).

Step 3: rear = rear + 1.

Step 4: queue[rear] = item.

Step 5: Stop.

Delete (DEQUEUE)

Algorithm: DELETE(queue)

Step 1: If front == -1 or front > rear, print "Queue Underflow" and stop.

Step 2: item = queue[front].

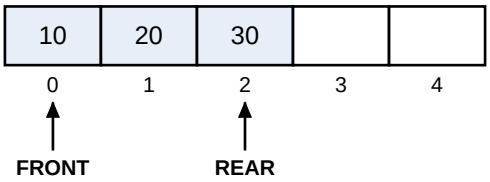
Step 3: front = front + 1.

Step 4: Return item. Stop.

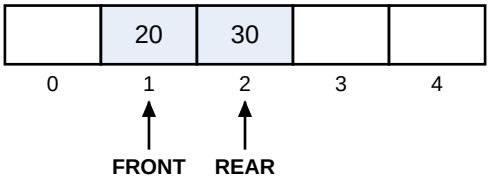
```
#include <stdio.h>
#define MAX 5
int queue[MAX], front = -1, rear = -1;

void enqueue(int item) {
    if (rear == MAX - 1) { printf("Queue Overflow\n"); return; }
    if (front == -1) front = 0;
    queue[++rear] = item;
}
int dequeue() {
    if (front == -1 || front > rear) { printf("Queue Underflow\n"); return -1; }
    return queue[front++];
}
void display() { int i; for (i = front; i <= rear; i++) printf("%d ", queue[i]); }
```

Trace (size 5):



After enqueue(10), enqueue(20), enqueue(30): front = 0, rear = 2



After dequeue() which removes 10: front = 1, rear = 2

Drawback of a linear queue

- After some deletions the empty cells at the left side cannot be reused even though space is free. Queue shows "Overflow" when rear = MAX - 1. This wastes memory.
- The solution is a **circular queue**.

Queue using a linked list

```
struct node { int data; struct node *next; } *front = NULL, *rear = NULL;

void enqueue(int item) {
    struct node *n = (struct node *) malloc(sizeof(struct node));
    n->data = item; n->next = NULL;
    if (rear == NULL) front = rear = n;     // first node
    else { rear->next = n; rear = n; }      // add at the end
}
int dequeue() {
    struct node *t = front; int item;
    if (front == NULL) { printf("Underflow"); return -1; }
    item = front->data; front = front->next;
    if (front == NULL) rear = NULL;         // queue became empty
    free(t); return item;
}
```

4.4 Circular queue

In a **circular queue** the last position is connected back to the first position to form a circle. When rear reaches the last index, it goes to index 0 if that cell is empty. This reuses the empty space and removes the drawback of the linear queue.

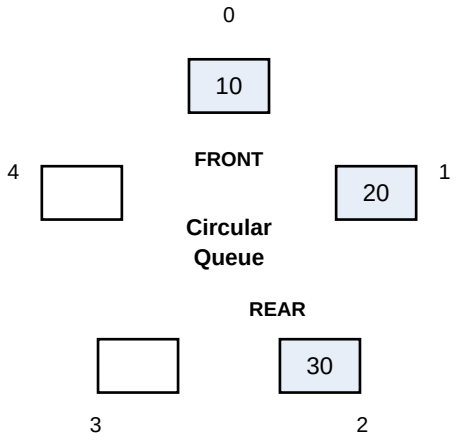


Fig 2.4: Circular queue of size 5 (front = 0, rear = 2)

Important formulas

- **rear = (rear + 1) % MAX** (move rear forward in a circle)
- **front = (front + 1) % MAX** (move front forward in a circle)
- **Queue is full** when (rear + 1) % MAX == front
- **Queue is empty** when front == -1

Algorithm: CIRCULAR INSERT(item)

Step 1: If (rear + 1) % MAX == front, print "Queue Overflow" and stop.

Step 2: If front == -1, set front = 0 and rear = 0. Else rear = (rear + 1) % MAX.

Step 3: queue[rear] = item. Stop.

Algorithm: CIRCULAR DELETE()

Step 1: If front == -1, print "Queue Underflow" and stop.

Step 2: item = queue[front].

Step 3: If front == rear, set front = rear = -1 (queue becomes empty). Else front = (front + 1) % MAX.

Step 4: Return item. Stop.

```
void cenqueue(int item) {
    if ((rear + 1) % MAX == front) { printf("Overflow"); return; }
    if (front == -1) front = rear = 0;
    else rear = (rear + 1) % MAX;
    queue[rear] = item;
}
int cdequeue() {
    int item;
    if (front == -1) { printf("Underflow"); return -1; }
    item = queue[front];
    if (front == rear) front = rear = -1;
    else front = (front + 1) % MAX;
    return item;
}
```

Applications of circular queue: CPU scheduling (round-robin), memory management, traffic light control, buffers (keyboard buffer, streaming data).

4.5 Dequeue (Double Ended Queue)

A **deque** allows insertion and deletion at **both ends** (front and rear). So it works like both a stack and a queue. Operations: insert_front, insert_rear, delete_front, delete_rear.

Type	Meaning
Input restricted deque	Insertion is allowed at only one end, deletion at both ends.
Output restricted deque	Deletion is allowed at only one end, insertion at both ends.

Applications: undo/redo operations, palindrome checking, sliding window problems, job scheduling algorithms.

4.6 Priority queue

In a **priority queue**, every element has a **priority**. The element with the **highest priority is removed first**, irrespective of the order of insertion. If two elements have the same priority, they are served in FIFO order.

- **Ascending priority queue:** the smallest number has the highest priority (removed first).
- **Descending priority queue:** the largest number has the highest priority.
- **Implementation:** using an array/linked list (keep sorted by priority), or using a heap (best, O(log n)).

Example (smaller number = higher priority). Insert (Task, priority): (A,3), (B,1), (C,2). Order of deletion: **B (1) -> C (2) -> A (3)**, even though A was inserted first.

Task	A	B	C
Priority	3	1	2
Deletion order	3rd	1st	2nd

Applications: CPU / process scheduling, hospital emergency (serious patients first), Dijkstra's shortest path, Huffman coding, network packet handling.

Queue type	Applications
Linear queue	Printer spooling, ticket counters, call centre
Circular queue	Round-robin CPU scheduling, memory buffers, traffic lights
Deque	Undo/redo, palindrome check, browser history

Queue type	Applications
Priority queue	Process scheduling, emergency services, Dijkstra's algorithm

5. Quick Revision - CO2

Remember these points

- Stack = LIFO, operations at TOP. PUSH: top++ then store. POP: take then top--. Overflow: top == MAX-1; Underflow: top == -1.
- Infix: A+B, Prefix: +AB, Postfix: AB+. Postfix evaluation uses a stack: operand -> push, operator -> pop two, compute, push.
- Priority: ^ (highest), * / %, + - (lowest). Brackets are pushed and popped when) comes.
- Recursion needs a base case. Function calls are stored on the call stack. Hanoi needs 2^n - 1 moves.
- Queue = FIFO. Insert at REAR, delete at FRONT. Linear queue wastes space; circular queue uses (rear+1)%MAX.
- Deque = both ends. Priority queue = highest priority first.

Practice questions

1. Define stack. Explain PUSH and POP with algorithms and an array representation.
2. What are polish notations? Explain with examples.
3. Convert A + B * C - D / E to postfix and (A + B) * C to prefix.
4. Evaluate the postfix expression 6 2 / 3 - 4 2 * + (Answer: 8).
5. Explain recursion. Write a recursive function for factorial and show the call stack for n = 4.
6. Explain the Tower of Hanoi problem and list the moves for 3 disks.
7. Define queue. Explain insert and delete operations using an array.
8. Explain circular queue and priority queue with one application of each.
9. Explain deque and its types.